

## **ConcurrentChat: Initial Planning and Design**

Group Project Deliverable 1 (Friday) - Option 2: Simple Chat Application

Languages: Rust and Go

Rahul Solanki, Krinal Soni, and Bijay Raj KC

University of the Cumberland

MSCS-632-M30: Advanced Programming Language

Jay Thom

October 2, 2026

## **ConcurrentChat: Initial Planning and Design**

### **Project Overview**

Our team is assigned Option 2, a simple chat application, implemented in Rust and Go. The application, which we call ConcurrentChat, is a text-based chat room in which several simulated users send messages concurrently to a central hub that owns the message history and answers queries against it. We will build it twice, once per language, with identical external behaviour.

The two languages are well matched to this application and to the purpose of the assignment. Both compile to native code and both were designed for concurrent systems, so neither has an unfair advantage on raw speed, and the comparison stays focused on design rather than on runtime overhead. At the same time they reach concurrency from opposite directions. Go is built on communicating sequential processes, in which independent processes exchange values over channels instead of sharing memory (Hoare, 1978; Donovan & Kernighan, 2015). Rust reaches a similar place through its type system, using ownership and borrowing to prove at compile time that data is not aliased across threads (Jung et al., 2021). A chat hub is a small enough problem to implement well in three days, and a demanding enough one that these differences become visible in ordinary code rather than in contrived examples.

### **Application Design**

#### ***Core Requirements***

Table 1 maps the three required features onto the design. Each is satisfied identically in both languages so that behaviour never differs between implementations.

**Table 1***Required Features and How the Design Satisfies Them*

| Requirement                                      | Design element                                                                                                       |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Simulate multiple users sending messages         | One independent worker per user (async task in Rust, goroutine in Go) posting to a shared hub                        |
| Store message history with timestamp and user ID | Each Message carries id, timestamp, user_id, username and a kind-specific payload; the hub assigns id and timestamp  |
| Filtering and search by user or keyword          | Hub queries History(limit), ByUser(name) and Search(keyword), plus a Stats summary; all matching is case-insensitive |

**Architecture**

Both implementations follow a single-owner hub design. Every user worker holds only a sending handle; the message history itself is owned by one hub worker and is never shared. Commands travel to the hub over a channel, and any command expecting an answer carries the channel the answer returns on.

**Figure 1. Architecture shared by both implementations.**

The consequence of this design is that neither implementation needs a mutex. What we expect to differ, and what we intend to document, is how each language establishes that guarantee. In Rust the hub takes ownership of the history when the task is spawned, so the compiler can prove no other task reaches it. In Go the same property holds only because we follow the convention of not passing the slice elsewhere; the compiler does not check it.

**Component Breakdown****Table 2***Components and Their Locations in Each Implementation*

| Component    | Responsibility                                | Rust         | Go                     |
|--------------|-----------------------------------------------|--------------|------------------------|
| Domain model | Message representation, rendering, predicates | src/model.rs | internal/chat/model.go |

| Component | Responsibility                                | Rust                        | Go                          |
|-----------|-----------------------------------------------|-----------------------------|-----------------------------|
| Hub       | Owens history; applies posts; answers queries | src/hub.rs                  | internal/chat/hub.go        |
| Driver    | CLI modes: demo, bench, REPL                  | src/main.rs                 | main.go                     |
| Tests     | Model predicates and hub behaviour            | #[cfg(test)] in<br>model.rs | internal/chat/model_test.go |

### ***Message Model and Command Set***

A message is one of five kinds: a room message, a direct message to a named recipient, a join, a leave, or a system announcement. Only the first, second and fifth carry text, and only a direct message has a recipient. This is the single most interesting modelling decision in the project, because Rust can express it directly as an algebraic data type while Go has no equivalent construct and must approximate it.

The hub accepts four commands: Post, Ask (carrying a query), Stats and Shutdown. Queries are History with a limit, ByUser with a name, and Search with a keyword. Commands must be processed in the order they are accepted, so that a query issued after a post observes that post.

### ***Command-Line Interface***

Both implementations expose the same three modes, which keeps demonstration and measurement symmetrical. The demo mode runs a scripted concurrent session and then exercises filtering, search and the summary. The bench mode starts a configurable number of concurrent producers and reports throughput. With no arguments the program is an interactive prompt supporting /send, /dm, /history, /user, /search, /stats and /quit.

### ***Anticipated Language Differences***

Table 3 records what we expect to find before writing the code. The Day 3 comparison report will revisit each row and state where we were right and where we were wrong, which is more useful than only reporting conclusions that happened to hold.

**Table 3***Expected Differences, Recorded Before Implementation*

| Area           | Expectation                                                                                                                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sum types      | Rust enums carry a payload per variant and match is exhaustive; Go must approximate this with a tag plus unused fields or an interface plus a type switch, neither checked for exhaustiveness |
| Concurrency    | Go goroutines read as ordinary synchronous code; Rust async requires explicit await and a runtime, so we expect Go to be easier to write and Rust to encode more in its types                 |
| Error handling | Rust Result must be consumed or the compiler warns; a Go error can be discarded with a single underscore                                                                                      |
| Memory         | Neither requires manual freeing; Rust frees at scope exit, Go uses a garbage collector, which we expect to cost memory under load                                                             |
| Modularity     | Rust modules with explicit pub versus Go packages with capitalisation-based export                                                                                                            |
| Performance    | Within the same order of magnitude, with Rust ahead because it has no collector and no interface dispatch in the hot path                                                                     |

***Risks Identified Up Front***

- Ordering. If commands travel on separate channels, Go's select chooses at random among ready cases, so a query could be served before queued posts are applied.  
Mitigation: a single ordered command channel in both implementations.
- Demo determinism. Concurrent interleaving differs between runs, so tests must assert on counts and filter results rather than on exact message order.
- Toolchain parity. Timings are only comparable if both run on the same machine.

Mitigation: one WSL2 Ubuntu 24.04 environment with rustc 1.95.0 and Go 1.27.1 for all builds and measurements.

**Task Assignment**

Roles are assigned so that each member owns one language or the cross-cutting analysis, and so that no member is blocked waiting on another. All three members participate in the presentation.

**Table 4***Role Assignment*

| Member        | Primary responsibility                            | Also contributes                                      |
|---------------|---------------------------------------------------|-------------------------------------------------------|
| Rahul Solanki | Rust implementation: model, hub, async tasks, CLI | Benchmark harness and measurement; repository setup   |
| Krinal Soni   | Go implementation: model, hub, goroutines, CLI    | Unit tests, go vet and race-detector runs             |
| Bijay Raj KC  | Shared specification and documentation            | Comparison report, screenshots, presentation assembly |

*Note.* The shared specification is written first and agreed by all three members, so that the two implementations are comparable rather than merely similar.

**Timeline****Table 5***Three-Day Milestone Plan*

| When   | Milestone                                              | Owner             |
|--------|--------------------------------------------------------|-------------------|
| Fri    | Shared specification, architecture, command set agreed | Bijay (lead), all |
| Fri    | Three repositories created; planning report submitted  | Rahul             |
| Sat AM | Domain model and hub compiling in both languages       | Rahul, Krinal     |
| Sat PM | Concurrent demo working end to end in both             | Rahul, Krinal     |
| Sat PM | Core-functionality report with screenshots             | Bijay             |
| Sun AM | Filtering, search, direct messages, statistics, REPL   | Rahul, Krinal     |
| Sun AM | Unit tests, go vet and race detector passing           | Krinal            |
| Sun PM | Benchmark runs and comparison report                   | Rahul, Bijay      |
| Sun PM | Presentation slides and rehearsal                      | All               |

**Repository**

The planning materials for this deliverable, including the full shared specification and the acceptance checklist, are in the public repository at <https://github.com/iknalos/MSCS-632-M30-Group-D1>. Implementation is tracked in separate repositories for Day 2 and Day 3, which are listed in that repository's README.

## References

- Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.
- Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, 21(8), 666-677. <https://doi.org/10.1145/359576.359585>
- Jung, R., Jourdan, J.-H., Krebbers, R., & Dreyer, D. (2021). Safe systems programming in Rust. *Communications of the ACM*, 64(4), 144-152. <https://doi.org/10.1145/3418295>
- Klabnik, S., & Nichols, C. (2023). *The Rust programming language* (2nd ed.). No Starch Press.
- Sebesta, R. W. (2019). *Concepts of programming languages* (12th ed.). Pearson.